



SW Pilot 학습보고서 <과정 5>

파이썬 5팀 황문규

문제 1. 통신장비와 통신 - 코드

```
1 # 과정 5 - (문제1) "통신장비와 통신" (서버)
2
3 import socket
4
5 HOST = socket.gethostname()
6 PORT = 9999
7
8 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9 print('소켓 생성완료')
10
11 server_socket.bind((HOST, PORT))
12
13 # 커넥션 3개로 제한
14 server_socket.listen(3)
15
16 print(f'호스트 이름: {HOST}')
17 print(f'{PORT}번 포트에서 연결 대기 중...')
18
19 # [보너스과제] - 챗봇용 응답 키워드
20 chatbot_responses = {
21     'hello': '안녕하세요! 반가워요!',
22     'name': '저는 예코봇입니다.',
23     'weather': '오늘 날씨는 맑음입니다.'
24 }
25
26 while True:
27     client_socket, addr = server_socket.accept()
28     print('클라이언트와 연결 되었습니다.')
29
30     while True:
31         data = client_socket.recv(1024).decode('utf-8')
32         if not data:
33             break
34
35         print(f'클라이언트로부터 받은 메시지: {data}')
36
37         if data.lower() == 'quit':
38             client_socket.send('연결을 종료합니다.'.encode('utf-8'))
39             break
40
41         response = chatbot_responses.get(data.strip(), data) # echo or chatbot
42         client_socket.send(response.encode('utf-8'))
43
44     client_socket.close()
45     print('클라이언트 연결 종료')
46
```

```
1 # 과정 5 - (문제1) "통신장비와 통신" (클라이언트)
2
3 import socket
4
5 def main():
6     # 서버 주소 및 포트 설정
7     host = socket.gethostname()
8     port = 9999
9
10    # 소켓 생성 및 연결
11    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12    try:
13        client_socket.connect((host, port))
14        print('클라이언트와 연결 되었습니다.')
15
16        while True:
17            # 사용자 입력
18            message = input('메시지 입력 (종료하려면 "quit"): ')
19            client_socket.send(message.encode('utf-8'))
20
21            if message.lower() == 'quit':
22                print('연결을 종료합니다.')
23                break
24
25            # 서버로부터 회신 수신
26            response = client_socket.recv(1024).decode('utf-8')
27            print(f'서버 응답: {response}')
28
29    except ConnectionRefusedError:
30        print('서버에 연결할 수 없습니다. 서버가 실행 중인지 확인하세요.')
31
32    finally:
33        client_socket.close()
34
35 if __name__ == '__main__':
36     main()
```

문제 1. 통신장비와 통신 - 구현화면

```
swpilotPyhon5 — python src/Process5/5-1/server.py — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-1/server.py]
소켓 생성 완료
호스트 이름 : hwangmungyuui-MacBookPro.local
9999번 포트에서 연결 대기 중 ...
클라이언트와 연결 되었습니다.
클라이언트로부터 받은 메시지 : hello
클라이언트로부터 받은 메시지 : weather
클라이언트로부터 받은 메시지 : na,me
클라이언트로부터 받은 메시지 : name
클라이언트로부터 받은 메시지 : 안녕
클라이언트로부터 받은 메시지 : sw파일럿 파이썬 과정 파이썬 5팀 황문규
클라이언트로부터 받은 메시지 : quit
클라이언트 연결 종료
클라이언트와 연결 되었습니다.
클라이언트로부터 받은 메시지 : 안녕
클라이언트로부터 받은 메시지 : 인사
클라이언트로부터 받은 메시지 : quit
클라이언트 연결 종료
[]
```

```
swpilotPyhon5 — -zsh — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-1/client.py]
클라이언트와 연결 되었습니다.
메시지 입력 (종료하려면 "quit"): hello
서버 응답 : 안녕하세요! 반가워요!
메시지 입력 (종료하려면 "quit"): weather
서버 응답 : 오늘 날씨는 맑음입니다.
메시지 입력 (종료하려면 "quit"): na,me
서버 응답 : na,me
메시지 입력 (종료하려면 "quit"): name
서버 응답 : 저는 예코봇입니다.
메시지 입력 (종료하려면 "quit"): 안녕
서버 응답 : 안녕
메시지 입력 (종료하려면 "quit"): sw파일럿 파이썬 과정 파이썬 5팀 황문규
서버 응답 : sw파일럿 파이썬 과정 파이썬 5팀 황문규
메시지 입력 (종료하려면 "quit"): quit
연결을 종료합니다.
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-1/client.py]
클라이언트와 연결 되었습니다.
메시지 입력 (종료하려면 "quit"): 안녕
서버 응답 : 안녕
메시지 입력 (종료하려면 "quit"): 인사
서버 응답 : 인사
메시지 입력 (종료하려면 "quit"): quit
연결을 종료합니다.
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % ]
```

문제 2. 통신장비에 채팅 기능 추가 - 코드

```
1 # 과정 5 - (문제2) "통신장비에 채팅 기능 추가" (서버)
2
3 import socket
4 import threading
5
6 HOST = socket.gethostname()
7 PORT = 9999
8
9 clients = {}
10 lock = threading.Lock()
11
12 def broadcast(message, sender_name=None):
13     with lock:
14         for name, conn in clients.items():
15             if name != sender_name:
16                 try:
17                     conn.sendall(message.encode('utf-8'))
18                 except:
19                     pass
20
21 def handle_client(conn, addr):
22     try:
23         conn.send('이름을 입력하세요: '.encode('utf-8'))
24         name = conn.recv(1024).decode('utf-8').strip()
25
26         with lock:
27             clients[name] = conn
28             welcome_msg = f'{name}님이 입장하셨습니다.\n'
29             broadcast(welcome_msg)
30             print(f'{name}님 연결됨. {addr}')
31
32         while True:
33             msg = conn.recv(1024).decode('utf-8').strip()
34             if not msg:
35                 continue
36             if msg == '/종료':
37                 with lock:
38                     del clients[name]
39                     exit_msg = f'{name}님이 퇴장하셨습니다.\n'
40                     print(exit_msg.strip())
41                     broadcast(exit_msg)
42                     break
43             elif msg.startswith('/귓속말'):
44                 parts = msg.split(' ', 2)
45                 if len(parts) < 3:
46                     conn.send('사용법: /귓속말 대상이름 메시지\n'.encode('utf-8'))
47                 else:
48                     target, whisper = parts[1], parts[2]
49                     with lock:
50                         target_conn = clients.get(target)
51                         if target_conn:
52                             whisper_msg = f'{name} > {whisper}\n'
53                             try:
54                                 target_conn.send(whisper_msg.encode('utf-8'))
55                             except:
56                                 pass
57                         else:
58                             conn.send('해당 사용자가 없습니다.\n'.encode('utf-8'))
59             else:
60                 broadcast_msg = f'{name} > {msg}\n'
61                 print(broadcast_msg.strip())
62                 broadcast(broadcast_msg, sender_name=name)
63     except ConnectionResetError:
64         pass
65     finally:
66         with lock:
67             if name in clients:
68                 del clients[name]
69             disconnect_msg = f'{name}님이 연결을 종료하셨습니다.\n'
70             print(disconnect_msg.strip())
71             broadcast(disconnect_msg)
72             conn.close()
73
74 def main():
75     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
76     server_socket.bind((HOST, PORT))
77     server_socket.listen(5)
78     print(f'서버 시작됨. 호스트: {HOST}, 포트: {PORT}')
79
80     while True:
81         conn, addr = server_socket.accept()
82         threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()
83
84 if __name__ == '__main__':
85     main()
```

```
1 # 과정 5 - (문제2) "통신장비에 채팅 기능 추가" (클라이언트)
2
3 import socket
4 import threading
5 import sys
6
7 HOST = socket.gethostname()
8 PORT = 9999
9
10 def receive_messages(sock):
11     while True:
12         try:
13             data = sock.recv(1024)
14             if not data:
15                 break
16             message = data.decode('utf-8')
17             sys.stdout.write('\r' + ' ' * 80 + '\r') # 현재 줄 클리어
18             print(message.strip())
19             sys.stdout.write(f'{username}> ')
20             sys.stdout.flush()
21         except:
22             break
23
24 def main():
25     global username
26
27     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28     client_socket.connect((HOST, PORT))
29
30     # 서버에서 이름 요청 메시지 수신
31     prompt = client_socket.recv(1024).decode('utf-8')
32     print(prompt, end='')
33     username = input()
34     client_socket.sendall(username.encode('utf-8'))
35
36     # 메시지 수신 스레드 시작
37     threading.Thread(target=receive_messages, args=(client_socket,), daemon=True).start()
38
39     while True:
40         sys.stdout.write(f'{username}> ')
41         sys.stdout.flush()
42         msg = input()
43         if msg == '/종료':
44             client_socket.sendall(msg.encode('utf-8'))
45             break
46         client_socket.sendall(msg.encode('utf-8'))
47
48     client_socket.close()
49
50 if __name__ == '__main__':
51     main()
```

문제 2. 통신장비에 채팅 기능 추가 - 구현화면

```
swpilotPyhon5 — python src/Process5/5-2/server.py — 100x30
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-2/server.py
서버 시작됨. 호스트: hwangmungyuui-MacBookPro.local, 포트: 9999
alex님 연결됨. (('127.0.0.1', 58147))
James님 연결됨. (('127.0.0.1', 58148))
Korea님 연결됨. (('127.0.0.1', 58152))
alex > Hi Everyone
James > Hi
Korea > Oh Hi
alex > kkk
alex > /quit Korea At home
Korea > goood
alex > bye
alex님이 퇴장하셨습니다.
alex님이 연결을 종료하셨습니다.
James > byebye
James님이 퇴장하셨습니다.
James님이 연결을 종료하셨습니다.
Korea > T_T
Korea님이 퇴장하셨습니다.
Korea님이 연결을 종료하셨습니다.
```

```
swpilotPyhon5 — zsh — 100x30
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-2/client.py
이름을 입력하세요: James
James> James님이 입장하셨습니다.
Korea님이 입장하셨습니다.
alex> Hi Everyone
James> Hi
Korea> Oh Hi
[귓속말] alex> hmm How have you been?
James> /귓속말 alex Im Good~~
alex> kkk
alex> /quit Korea At home
Korea> goood
alex> bye
alex님이 퇴장하셨습니다.
alex님이 연결을 종료하셨습니다.
James> π byebye
James> /종료
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % )
```

```
swpilotPyhon5 — zsh — 100x30
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-2/client.py
이름을 입력하세요: alex
alex님이 입장하셨습니다.
James님이 입장하셨습니다.
Korea님이 입장하셨습니다.
alex> Hi Everyone
James> Hi
Korea> Oh Hi
alex> /귓속말 James hmm How have you been?
[귓속말] James> Im Good~~
alex> kkk
[귓속말] Korea> where are you?
alex> /귓속말 Korea At home
alex> /귓속말 Korea At home
Korea> goood
alex> bye
alex> /종료
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % )
```

```
swpilotPyhon5 — zsh — 100x24
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process5/5-2/client.py
이름을 입력하세요: Korea
Korea님이 입장하셨습니다.
alex> Hi Everyone
James> Hi
Korea> Oh Hi
alex> kkk
Korea> /귓속말 alex where are you?
alex> /quit Korea At home
[귓속말] alex> At home
Korea> goood
alex> bye
alex님이 퇴장하셨습니다.
alex님이 연결을 종료하셨습니다.
James> byebye
James님이 퇴장하셨습니다.
James님이 연결을 종료하셨습니다.
Korea> T_T
Korea> /종료
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % )
```

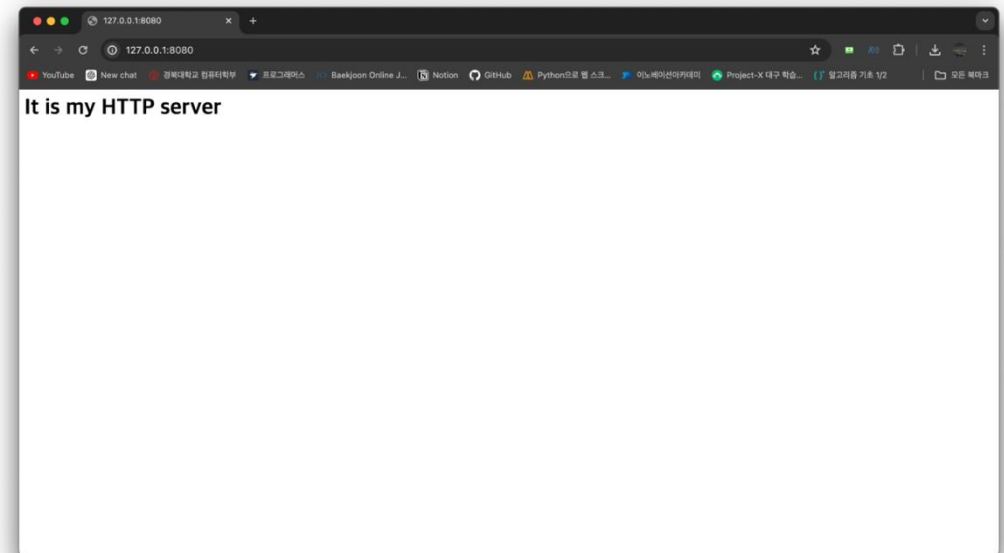
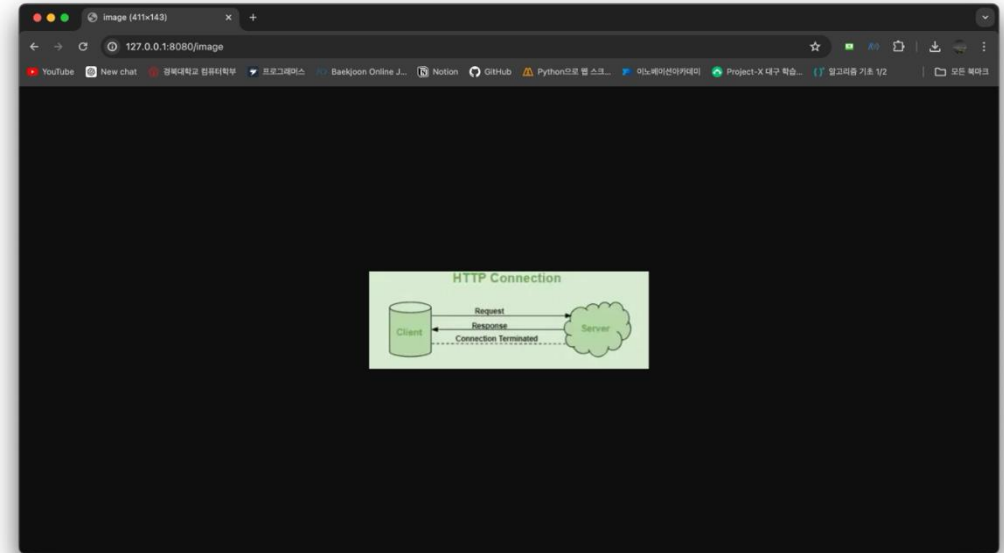
문제 3. 나 혼자 보는 웹서버 - 코드

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2
3 class MyHTTPRequestHandler(BaseHTTPRequestHandler):
4     def do_GET(self):
5         try:
6             # 요청 경로를 기반으로 파일 경로 결정
7             if self.path == '/image':
8                 self.path = '/http_image.jpg' # 보너스 과제 - 이미지 출력 : 경로에 /image 추가해서 해당 링크로 접속 시 이미지 출력
9
10            with open('.' + self.path, 'rb') as file:
11                # 파일 확장자에 따라 Content-type 설정
12                if self.path.endswith('.jpg'):
13                    self.send_response(200)
14                    self.send_header('Content-type', 'image/jpeg')
15                    self.end_headers()
16                    # 파일 내용 읽어서 클라이언트에게 전송
17                    self.wfile.write(file.read())
18
19            else:
20                # 다른 추가 링크가 없을 시 원래의 과제 수행
21                self.send_response(200)
22                self.send_header('Content-type', 'text/html; charset=utf-8')
23                self.end_headers()
24                # 클라이언트에게 보낼 HTML 문서
25                html_content = """
26                <html>
27                <body>
28                <h1>It is my HTTP server</h1>
29                </body>
30                </html>
31                """
32                self.wfile.write(html_content.encode('utf-8'))
33
34            except IOError:
35                self.send_error(404, 'File Not Found: %s' % self.path)
```

```
1
2 def run(server_class=HTTPServer, handler_class=MyHTTPRequestHandler, port=8080):
3     server_address = ('', port)
4     httpd = server_class(server_address, handler_class)
5
6     print(f'Starting HTTP server on port {port}...')
7     try:
8         httpd.serve_forever()
9     except KeyboardInterrupt:
10        pass
11
12    httpd.server_close()
13    print('Stopping HTTP server...')
14
15 if __name__ == '__main__':
16    run()
```


문제 3. 나 혼자 보는 웹서버 - 구현화면

```
Process5 -- zsh -- 109x29
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level3/myhttpServer.py
Starting HTTP server on port 8080...
127.0.0.1 - - [20/May/2024 12:43:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:08] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:29] "GET /image HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:29] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:46] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:46] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:43:59] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:44:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/May/2024 12:44:02] "GET /favicon.ico HTTP/1.1" 200 -
^CStopping HTTP server...
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 %
```



문제 4. 조금 더 나은 웹서버 - 코드 (서버)

```
1 import http.client
2 import json
3 from http.server import BaseHTTPRequestHandler, HTTPServer
4 from datetime import datetime
5
6 # HTTP 서버 설정
7 PORT = 8080
8 server_address = ('', PORT)
9
10 # index.html 파일 경로
11 HTML_FILE_PATH = 'Level4/index.html'
12
13 # IPinfo.io API endpoint
14 IPINFO_API_HOST = 'ipinfo.io'
15 IPINFO_API_PATH = '/{ip}/json'
16
17 # HTTP 요청을 처리할 핸들러 클래스 정의
18 class MyHTTPRequestHandler(BaseHTTPRequestHandler):
19     def do_GET(self):
20         # 요청 받은 시간
21         current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
22         # 클라이언트의 IP 주소
23         client_address = self.client_address[0]
24         # IP 주소를 기반으로 위치 정보 조회
25         location_info = self.get_location_info(client_address)
26
27         # 서버 측에서 접속 정보 출력
28         print(f'[{current_time}] 클라이언트 접속: {client_address}, 접속 위치 : {location_info}')
29
30         # 응답 상태코드 200과 헤더 설정
31         self.send_response(200)
32         self.send_header('Content-type', 'text/html; charset=utf-8')
33         self.end_headers()
34
35         # index.html 파일 읽기
36         with open(HTML_FILE_PATH, 'rb') as file:
37             html_content = file.read()
38
39         # HTML 문서를 클라이언트에 전송
40         self.wfile.write(html_content)
41
```

```
1
2 def get_location_info(self, ip_address):
3     try:
4         # 예외 처리: localhost에 접속한 경우
5         if ip_address == '127.0.0.1' or ip_address == '::1':
6             return 'localhost'
7
8         # HTTP 연결 설정
9         conn = http.client.HTTPSConnection(IPINFO_API_HOST)
10        url = IPINFO_API_PATH.format(ip=ip_address)
11
12        # HTTP GET 요청 전송
13        conn.request('GET', url)
14
15        # HTTP 응답 가져오기
16        response = conn.getresponse()
17
18        # 응답 데이터 읽기
19        data = response.read().decode('utf-8')
20
21        # JSON 데이터 파싱
22        location_data = json.loads(data)
23
24        # 위치 정보 추출
25        country = location_data.get('country', 'Unknown Country')
26        city = location_data.get('city', 'Unknown City')
27
28        location_info = f'{country}, {city}'
29
30        # 연결 닫기
31        conn.close()
32
33        return location_info
34
35    except Exception as e:
36        print(f'Error fetching location information: {e}')
37        return 'Location not available'
38
39
40 # HTTP 서버 실행 함수
41 def run(server_class=HTTPServer, handler_class=MyHTTPRequestHandler, port=8080):
42     server_address = ('', port)
43     httpd = server_class(server_address, handler_class)
44
45     print(f'Starting HTTP server on port {port}...')
46     try:
47         httpd.serve_forever()
48     except KeyboardInterrupt:
49         pass
50
51     httpd.server_close()
52     print('Stopping HTTP server...')
53
54 # 메인으로 실행할 경우 HTTP 서버 시작
55 if __name__ == '__main__':
56     run()
57     print()
```

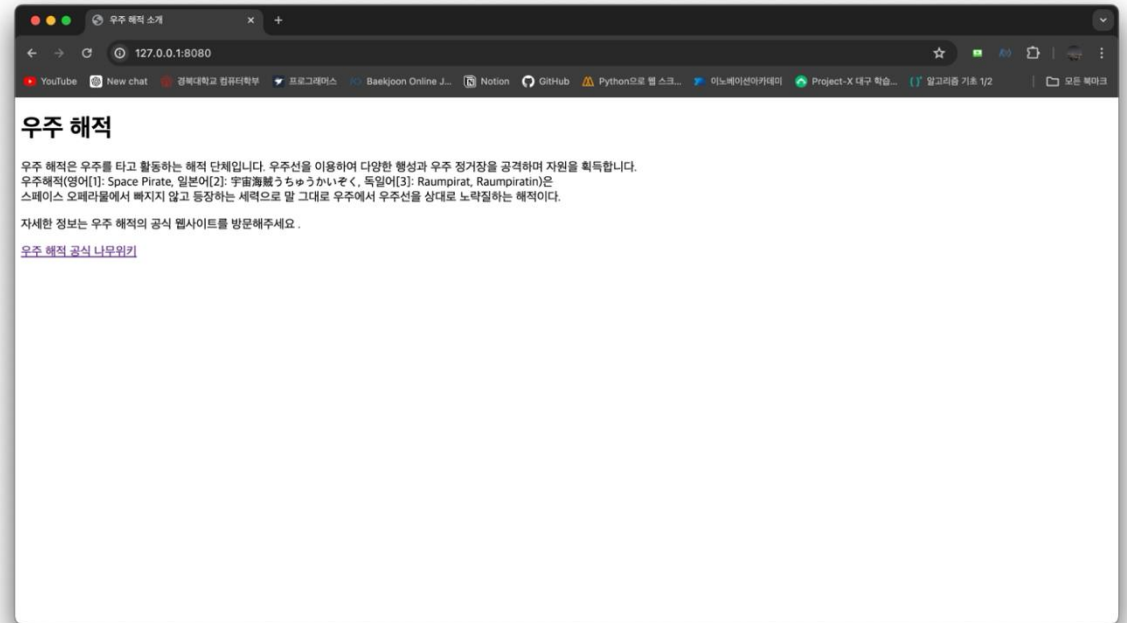

문제 4. 조금 더 나은 웹서버 - 코드 (index.html)



```
1  <!-- index.html -->
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>우주 해적 소개</title>
8  </head>
9  <body>
10     <h1>우주 해적</h1>
11     <p>우주 해적은 우주를 타고 활동하는 해적 단체입니다. 우주선을 이용하여 다양한 행성과 우주 정거장을 공격하며 자원을 획득합니다.<br>
12         우주해적(영어[1]: Space Pirate, 일본어[2]: 宇宙海賊うちゅうかいぞく, 독일어[3]: Raumpirat, Raumpiratin)은 <br>
13         스페이스 오페라물에서 빠지지 않고 등장하는 세력으로 말 그대로 우주에서 우주선을 상대로 노략질하는 해적이다.
14     </p>
15     <p>자세한 정보는 우주 해적의 공식 웹사이트를 방문해주세요 .</p>
16     <p><a href="https://namu.wiki/w/우주해적" target="_blank">우주 해적 공식 나무위키</a></p>
17 </body>
18 </html>
```

문제 4. 조금 더 나은 웹서버 - 구현화면

```
Process5 — Python Level4/mybetterhttpServer.py — 109x29
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level4/mybetterhttpServer.py
Starting HTTP server on port 8080...
[2024-05-20 12:47:58] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:47:58] "GET / HTTP/1.1" 200 -
[2024-05-20 12:48:00] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:00] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:48:06] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:06] "GET /image HTTP/1.1" 200 -
[2024-05-20 12:48:07] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:07] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:48:10] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:10] "GET / HTTP/1.1" 200 -
[2024-05-20 12:48:10] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:10] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:48:19] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:19] "GET / HTTP/1.1" 200 -
[2024-05-20 12:48:19] 클라이언트 접속 : 127.0.0.1, 접속 위치 : Localhost
127.0.0.1 - - [20/May/2024 12:48:19] "GET /favicon.ico HTTP/1.1" 200 -
```



문제 5. 접속한 사용자 정보를 살펴보자 - 코드 (1)

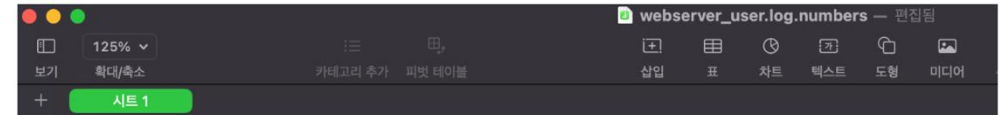
```
1 import http.client
2 import json
3 import csv
4 from http.server import BaseHTTPRequestHandler, HTTPServer
5 from datetime import datetime
6 import os.path
7
8 # HTTP 서버 설정
9 PORT = 8080
10 server_address = ('', PORT)
11
12 # index.html 파일 경로
13 HTML_FILE_PATH = 'Level4/index.html'
14
15 # IPinfo.io API endpoint
16 IPINFO_API_HOST = 'ipinfo.io'
17 IPINFO_API_PATH = '/ip/json'
18
19 # CSV 파일 이름
20 LOG_FILE = 'webserver_user_log.csv'
21
22 # 필드명 (헤더)
23 CSV_FIELDS = ['Timestamp', 'IP Address', 'Operating System', 'Browser']
24
25 # HTTP 요청을 처리할 핸들러 클래스 정의
26 class MyHTTPRequestHandler(BaseHTTPRequestHandler):
27
28     def do_GET(self):
29         # 요청 받은 시간
30         current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
31         # 클라이언트의 IP 주소
32         client_address = self.client_address[0]
33
34         # 클라이언트의 User Agent 정보
35         user_agent = self.headers.get('User-Agent', 'Unknown')
36
37         # 운영체제와 브라우저 정보 추출
38         operating_system, browser = self.parse_user_agent(user_agent)
39
40         # 서버 측에서 접속 정보 출력
41         print(f'[{current_time}] 클라이언트 접속: {client_address}, User-Agent: {user_agent}')
42
43         # CSV 파일에 접속 정보 기록
44         self.log_to_csv(current_time, client_address, operating_system, browser)
45
46         # IP 주소를 기반으로 위치 정보 조회
47         location_info = self.get_location_info(client_address)
48
49         # 응답 상태코드 200과 헤더 설정
50         self.send_response(200)
51         self.send_header('Content-type', 'text/html; charset=utf-8')
52         self.end_headers()
53
54         # index.html 파일 읽기
55         with open(HTML_FILE_PATH, 'rb') as file:
56             html_content = file.read()
57
58         # 위치 정보를 HTML 내용에 추가
59         html_content_with_location = html_content.replace(b'{{location}}', location_info.encode())
60
61         # HTML 문서를 클라이언트에 전송
62         self.wfile.write(html_content_with_location)
63
```

```
1     def parse_user_agent(self, user_agent):
2         # 간단한 예시로 운영체제와 브라우저 정보 추출
3         operating_system = 'Unknown OS'
4         browser = 'Unknown Browser'
5
6         if 'Windows' in user_agent:
7             operating_system = 'Windows'
8         elif 'Macintosh' in user_agent:
9             operating_system = 'Macintosh'
10        elif 'Linux' in user_agent:
11            operating_system = 'Linux'
12
13        if 'Chrome' in user_agent:
14            browser = 'Chrome'
15        elif 'Firefox' in user_agent:
16            browser = 'Firefox'
17        elif 'Safari' in user_agent:
18            browser = 'Safari'
19
20        return operating_system, browser
21
22    def log_to_csv(self, timestamp, ip_address, operating_system, browser):
23        try:
24            # 파일이 존재하지 않으면 헤더를 포함하여 새로 작성
25            if not os.path.exists(LOG_FILE):
26                with open(LOG_FILE, mode='w', newline='') as file:
27                    writer = csv.writer(file)
28                    writer.writerow(CSV_FIELDS)
29
30            # CSV 파일에 데이터 추가
31            with open(LOG_FILE, mode='a', newline='') as file:
32                writer = csv.writer(file)
33                writer.writerow([timestamp, ip_address, operating_system, browser])
34
35        except Exception as e:
36            print(f'Error writing to CSV file: {e}')
37
38    # HTTP 서버 실행 함수
39    def run(server_class=HTTPServer, handler_class=MyHTTPRequestHandler, port=8080):
40        server_address = ('', port)
41        httpd = server_class(server_address, handler_class)
42
43        print(f'Starting HTTP server on port {port}...')
44        try:
45            httpd.serve_forever()
46        except KeyboardInterrupt:
47            pass
48
49        httpd.server_close()
50        print('Stopping HTTP server...')
51
52    # 메인으로 실행할 경우 HTTP 서버 시작
53    if __name__ == '__main__':
54        run()
55
```

문제 5. 접속한 사용자 정보를 살펴보자 - 코드 (2)

```
1 # 보너스 과제 - webserver_user.log 파일을 읽어서 접속자들의 운영체제 및 웹 브라우저 별 통계를 내고 원 그래프로 시각화
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # CSV 파일 이름
6 LOG_FILE = 'webserver_user.log.csv'
7
8 def analyze_user_agents(csv_file):
9     try:
10         # CSV 파일 읽기
11         df = pd.read_csv(csv_file)
12
13         # 운영체제별 통계
14         os_counts = df['Operating System'].value_counts()
15
16         # 웹 브라우저별 통계
17         browser_counts = df['Browser'].value_counts()
18
19         # 시각화: 운영체제별 원 그래프
20         plt.figure(figsize=(10, 6))
21         plt.subplot(1, 2, 1)
22         os_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90)
23         plt.title('Operating System Distribution')
24         plt.ylabel('')
25
26         # 시각화: 웹 브라우저별 원 그래프
27         plt.subplot(1, 2, 2)
28         browser_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90)
29         plt.title('Web Browser Distribution')
30         plt.ylabel('')
31
32         # 그래프 출력
33         plt.tight_layout()
34         plt.show()
35
36     except Exception as e:
37         print(f'Error analyzing user agents: {e}')
38
39 # 메인 함수
40 def main():
41     analyze_user_agents(LOG_FILE)
42
43 if __name__ == '__main__':
44     main()
```

문제 5. 접속한 사용자 정보를 살펴보자 - 구현화면



```
Process5 — zsh — 109x29
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level5/userhttp.py
Starting HTTP server on port 8080...
[2024-05-20 12:53:59] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:53:59] "GET / HTTP/1.1" 200 -
[2024-05-20 12:53:59] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:53:59] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:54:00] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:00] "GET / HTTP/1.1" 200 -
[2024-05-20 12:54:00] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:00] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:54:09] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:09] "GET / HTTP/1.1" 200 -
[2024-05-20 12:54:09] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:09] "GET /favicon.ico HTTP/1.1" 200 -
[2024-05-20 12:54:13] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:13] "GET / HTTP/1.1" 200 -
[2024-05-20 12:54:13] 클라이언트 접속: 127.0.0.1, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
127.0.0.1 -- [20/May/2024 12:54:13] "GET /favicon.ico HTTP/1.1" 200 -
^CStopping HTTP server...
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 %
```

webservice_user.log					
Timestamp	IP Address	Operating System	Browser		
2024-05-18 15:51:29	127.0.0.1	Macintosh	Chrome		
2024-05-18 15:51:29	127.0.0.1	Macintosh	Chrome		
2024-05-18 15:51:36	127.0.0.1	Macintosh	Chrome		
2024-05-18 16:11:58	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:53:59	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:00	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:00	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:09	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:09	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:13	127.0.0.1	Macintosh	Chrome		
2024-05-20 12:54:13	127.0.0.1	Macintosh	Chrome		



문제 6. 지구와 연결된 네트워크 - 코드

```
1 import requests
2
3 def get_kbs_html() :
4     URL = 'https://news.kbs.co.kr/news/pc/main/main.html'
5
6     try :
7         response = requests.get(URL)
8         response.raise_for_status() # 요청이 성공적으로 이루어졌는지 확인
9
10        # HTTP 응답에서 HTML 문자열 가져오기
11        html = response.text
12        return html
13
14    except requests.exceptions.RequestException as e :
15        print(f'Error Getting KBS news : {e}')
16        return None
17
18 def extract_headlines(html) :
19     headlines = []
20
21     # 헤드라인 뉴스가 포함된 특정 패턴 찾기
22     start_pattern = """<div class="txt-wrapper">
23         <p class="title">"""
24     end_pattern = '</p>'
25
26     start_idx = html.find(start_pattern)
27
28     while start_idx != -1 :
29         end_idx = html.find(end_pattern, start_idx)
30
31         if end_idx != -1 :
32             headlines.append(html[start_idx + len(start_pattern):end_idx].strip().replace('<br>', ''))
33
34         start_idx = html.find(start_pattern, end_idx)
35
36     return headlines
37
38 def main() :
39     html_content = get_kbs_html()
40
41     headline_news = extract_headlines(html_content)
42
43     print('-----오늘의 헤드라인 뉴스-----')
44     for headline in headline_news :
45         print(headline)
46
47 if __name__ == '__main__' :
48     main()
```

```
1 # 보너스 과제 - 하루에 한 번 정기적으로 실행하는 방법 (code ver.)
2 import schedule
3 import time
4
5 def job():
6     print('Running KBS news crawling...')
7     html_content = get_kbs_html()
8     if html_content:
9         headline_news = extract_headlines(html_content)
10        print('-----오늘의 헤드라인 뉴스-----')
11        for headline in headline_news:
12            print(headline)
13
14 # schedule에 작업 등록 (매일 오전 9시에 실행)
15 schedule.every().day.at("09:00").do(job)
16
17 # 무한 루프로 스케줄러 실행
18 while True:
19     schedule.run_pending()
20     time.sleep(60) # 1분마다 체크하여 실행
21
22 # 보너스 과제 - 하루에 한 번 정기적으로 실행하는 방법 (using Windows ver.)
23 # Windows에서의 작업 스케줄러 설정
24 # 1. 작업 스케줄러 열기:
25 # 시작 메뉴에서 '작업 스케줄러'를 검색하여 실행합니다.
26 # 2. 새 작업 만들기:
27 # 오른쪽 창의 "기본 작업 만들기"를 클릭합니다.
28 # 3. 기본 정보 설정:
29 # 이름과 설명을 입력합니다. 예를 들어, "Daily KBS News Crawling".
30 # 4. 트리거 설정:
31 # "새로 만들기"를 클릭하고, 시작을 설정합니다.
32 # 원하는 트리거(일일, 특정 시간 등)를 선택하고 설정합니다.
33 # 5. 동작 설정:
34 # 동작 탭에서 "새로 만들기"를 클릭하여 실행할 프로그램을 설정합니다.
35 # 프로그램/스크립트란에는 Python 실행 파일의 경로를 입력합니다.
36 # 인수 추가란에는 실행할 스크립트 파일의 경로를 입력합니다.
37 # 6. 추가 설정:
38 # 필요에 따라 다른 설정(권한, 네트워크 조건 등)을 구성합니다.
```


문제 6. 지구와 연결된 네트워크 - 구현화면

```
Process5 — -zsh — 109x29
[(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level6/crawling_KBS.py]
-----오늘의 헤드라인 뉴스-----
"이란 대통령 헬기 추락 지점 발견...전원 사망 추정"
야 6당, 해병대원 특검법 수용 촉구...여야, '원 구성' 협상 시작
"전공의, 오늘까지 복귀해야...입학 전형 계획 신속 마무리"
'기획전' 한다더니 비용은 납품업체에...공정위, SSG닷컴·컬리 제재
데이터베이스 무단 복제해 헐값에 판매...대법 "저작권법 위반"
이원석 검찰총장 "김호중 등 의도적 사법 방해 적극 대응...구속사유에도 반영"
공수처 "김계한·박정훈 동시 소환 조율 중"...대질 신문 검토
북핵을 인정한 푸틴, 신냉전시대 한국 외교의 선택지는?
"야밤에 몰래 계단 깔더라"...구청에 보고한 정상적인 공사"
"그는 한미동맹을 모른다...거래의 대상일 뿐"
[(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % ]
```

문제 7. 정기적으로 배달되는 지구소식 - 코드

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 import requests
5
6 def get_kbs_headlines() :
7     URL = 'https://news.kbs.co.kr/news/pc/main/main.html'
8
9     try :
10         response = requests.get(URL)
11         response.raise_for_status() # 요청이 성공적으로 이루어졌는지 확인
12         # HTTP 응답에서 HTML 문자열 가져오기
13         html = response.text
14         soup = BeautifulSoup(html, 'html.parser')
15
16         with open('weather.txt','w') as file :
17             file.write(str(soup))
18
19         # <p> tag 중 class="title"인 요소 찾아서 list로 저장
20         headline_tags = soup.find_all('p', class_='title')
21
22         headline_list = []
23
24         for tag in headline_tags :
25             headline = tag.text.strip()
26             if headline :
27                 headline_list.append(headline)
28
29         # 헤드라인에 해당하지 않는 요소 제거
30         headline_list.pop()
31         headline_list.pop(0)
32
33         return headline_list
34
35 except requests.exceptions.RequestException as e :
36     print(f'Error Getting KBS news : {e}')
37     return None
38
```

```
39 # 보너스 과제 - 네이버 날씨 가져오기
40 def get_weather() :
41     URL = 'https://search.naver.com/search.naver?where=nexearch&sm=top_hyt&fbm=0&ie=utf8&query=%EB%82%A0%EC%94%A8'
42
43     try :
44         response = requests.get(URL)
45         response.raise_for_status()
46
47         html = response.text
48         soup = BeautifulSoup(html, 'html.parser')
49
50         weather_section = soup.find('section', class_='sc_new cs_weather_new _cs_weather')
51
52         location_info = weather_section.find('h2', class_='title')
53
54         today_weather_info = weather_section.find('div', class_='_today')
55
56         weekly_weather_info = weather_section.find_all('li', class_='week_item')
57
58         print('위치 : ', location_info.text)
59         print('오늘의 날씨 : ', today_weather_info.text.strip())
60         print('주간 날씨')
61         for weather in weekly_weather_info :
62             print(weather.text.strip())
63
64     except requests.exceptions.RequestException as e :
65         print(f'Error Getting Naver Weather : {e}')
66         return None
67
68
69 def main() :
70     headline_news = get_kbs_headlines()
71     print('-----오늘의 헤드라인 뉴스-----')
72     for headline in headline_news :
73         print(headline)
74
75     print()
76
77     print('-----오늘의 날씨-----')
78     # 보너스 과제 - 네이버 날씨 가져오기
79     weather = get_weather()
80
81
82 if __name__ == '__main__' :
83     main()
84
```

문제 7. 정기적으로 배달되는 지구소식 - 구현화면

```
Process5 - zsh - 150x47
[(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level7/crawling_KBS.py]
-----오늘의 헤드라인 뉴스-----
[속보] 로이터 "이란 당국자, 라이시 대통령·외무장관 사망 확인"
야 6당, 해병대원 특검법 수용 촉구...여야, '원 구성' 협상 시작
"전공의, 오늘까지 복귀해야...입학 전형 계획 신속 마무리"
'기획전' 한다더니 비용은 납품업체에...공정위, SSG닷컴·컬리 제재
데이터베이스 무단 복제해 헐값에 판매...대법 "저작권법 위반"
이원석 검찰총장 "김호중 등 의도적 사법 방해 적극 대응...구속 사유에도 반영"
공수처 "김계환·박정훈 등 소환 조율 중"...대질 신문 검토
이주호 "의대 증원 반영한 학칙 개정 마무리해달라...대학 의무"
의대협 "'기각' 판결 역사로 기록될 것"...투쟁 계속
북핵을 인정한 푸틴, 신냉전시대 한국 외교의 선택지는?
오늘부터 병·의원 신분증 확인 필수..."소모적 행정" 비판도
'김건희 여사 고가 가방 수수 의혹' 백은종 서울의소리 대표 오늘 소환 조사
"야밤에 몰래 계단 깔더라..."구형에 보고한 정상적인 공사"
'청년주택 드림 청약통장' 출시 3달 만에 가입자 100만 명 넘어
서울시, 외국 인재·기업 유치...외국인 주민 5개년 정책 발표
"그는 한미동맹을 모른다...거래의 대상일 뿐"
'생방송 중 전 연인 살해 예고'...40대 유튜버 '살인예비 혐의' 체포
라이칭더 타이완 총통 취임..."현상유지·양안 공동번영 입장 표명"
-----오늘의 날씨-----
위치 : 북구 복현1동
오늘의 날씨 : 구름많음 현재 온도 26.8° 어제보다 2.6° 낮아요 구름많음 체감 23.2° 습도 20% 서풍 2.4m/s
주간 날씨
오늘 5.20. 오전 10% 맑음 오후 20% 구름많음 최저기온 16° / 최고기온 30°
내일 5.21. 30% 흐림 20% 구름많음 최저기온 14° / 최고기온 25°
수 5.22. 0% 맑음 0% 맑음 최저기온 13° / 최고기온 30°
목 5.23. 10% 맑음 10% 맑음 최저기온 16° / 최고기온 32°
금 5.24. 10% 맑음 10% 맑음 최저기온 17° / 최고기온 29°
토 5.25. 30% 구름많음 30% 구름많음 최저기온 15° / 최고기온 26°
일 5.26. 30% 구름많음 40% 흐림 최저기온 14° / 최고기온 26°
월 5.27. 40% 흐림 40% 흐림 최저기온 16° / 최고기온 26°
화 5.28. 30% 구름많음 30% 구름많음 최저기온 15° / 최고기온 26°
수 5.29. 30% 구름많음 30% 구름많음 최저기온 15° / 최고기온 27°
[(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % ]
```

문제 8. 로그인을 넘어 - 코드

```
1 from selenium import webdriver
2 from selenium.webdriver.support.ui import WebDriverWait
3 from selenium.webdriver.support import expected_conditions as EC
4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.chrome.options import Options
6 from selenium.webdriver.chrome.service import Service
7 from selenium.webdriver.common.keys import Keys
8 from webdriver_manager.chrome import ChromeDriverManager
9
10 # 네이버 로그인 시 자동입력방지가 뜨지 않도록 하기 위해 사용하는 라이브러리들
11 import pyautogui
12 import pyperclip
13 import time
14
15 # naver ID & PW
16 naver_id = '*****'
17 naver_pw = '*****'
18
19 def bootChromeDriverandNaver():
20     options = Options()
21     options.add_experimental_option('detach', True) # 브라우저 바로 닫힘 방지
22     # options.add_experimental_option('excludeSwitches', ['enable-logging']) # 불필요한 메시지 제거
23
24     service = Service(ChromeDriverManager().install())
25
26     driver = webdriver.Chrome(service=service, options=options)
27
28     driver.get('https://naver.com')
29
30     return driver
31
32 def naverLogin(driver):
33     # login 창으로 이동
34     login_tab = driver.find_element(By.CSS_SELECTOR, 'a.MyView-module__link_login__HphWw')
35     login_tab.click()
36
37     # id 입력창
38     id = driver.find_element(By.CSS_SELECTOR, '#id')
39     id.click()
40     pyperclip.copy(naver_id)
41     pyautogui.keyDown("command")
42     pyautogui.press("v")
43     pyautogui.keyUp("command")
44     time.sleep(2)
45
46     # 비밀번호 입력창
47     pw = driver.find_element(By.CSS_SELECTOR, '#pw')
48     pw.click()
49     pyperclip.copy(naver_pw)
50     pyautogui.keyDown("command")
51     pyautogui.press("v")
52     pyautogui.keyUp("command")
53     time.sleep(1)
54
55     # login 버튼
56     btn_login = driver.find_element(By.CSS_SELECTOR, '#log\\.login')
57     btn_login.click()
58
59     return driver
60
```

```
1 def crawlNaver(driver):
2     # 특정 요소가 로드될 때까지 대기
3     wait = WebDriverWait(driver, 10)
4     wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, '#newsstand .MediaView-module__media_area__Z4js3')))
5
6     # 페이지가 완전히 로드된 후 약간의 추가 대기
7     time.sleep(2)
8
9     # 특정 클래스의 img 태그 선택
10    elements = driver.find_elements(By.CSS_SELECTOR, '#newsstand .MediaView-module__media_area__Z4js3 .MediaSubscriptionView-module__subscription_group__peb21 .MediaSubscriptionView-module__news_thumb__IA4y2 img')
11
12    # 각 img 태그의 alt 속성 값을 추출
13    news_titles = [element.get_attribute('alt') for element in elements]
14
15    # 추출된 텍스트 출력
16    for title in news_titles:
17        print(title)
18
19 # 뉴스 개개 - naverMail 목록 크롤링
20 def checkNaverMail(driver):
21     driver.get('https://mail.naver.com/v2/folders/0/all')
22
23     # 메일 목록이 완전히 로드될 때까지 대기
24     wait = WebDriverWait(driver, 10)
25     wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, 'a.mail_title_link > span.text'))))
26
27     # 보낸 사람 크롤링
28     sender_list = driver.find_elements(By.CSS_SELECTOR, 'div.mail_sender > button')
29     # 메일 목록 크롤링
30
31     mail_list = driver.find_elements(By.CSS_SELECTOR, 'a.mail_title_link > span.text')
32     for i in range(len(mail_list)):
33         print(f'{sender_list[i].text.replace('\n', ' ')} : {mail_list[i].text}')
34
35
36 if __name__ == '__main__':
37     chromeDriver = bootChromeDriverandNaver()
38     print('로그인 이전 추천 언론사')
39     crawlNaver(chromeDriver)
40     naverLogin(chromeDriver)
41     print('\n로그인 이후 추천 언론사')
42     crawlNaver(chromeDriver)
43     print('\n메일 목록')
44     checkNaverMail(chromeDriver)
```

문제 8. 로그인을 넘어 - 구현화면

```
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 % python3 Level8/crawling_KBS.py
/Users/hwangmungyu/Desktop/ProjectX/Process5/Level8/crawling_KBS.py:56: SyntaxWarning: invalid escape sequence '\.'
  btn_login = driver.find_element(By.CSS_SELECTOR, '#log\login')
로그인 이전 추천 언론사
한겨레
해럴드경제
서울신문
오마이뉴스
동아일보
국민일보
데일리안
MBC
JTBC
노컷뉴스
세계일보
코리아헤럴드
한국경제
시사IN
SBS
전북도민일보
전북일보
헬로디디
인벤
Arirang TV
이코노미조선
스타뉴스
스포츠헤드
디지털데일리
로그인 이후 추천 언론사
스포탈코리아
디지털타임스
서울경제
지디넷코리아
파이낸셜뉴스
MBC
프레시안
조선일보
블로터
스포츠포스트
국민일보
SBS
이데일리
세계일보
해럴드경제
전북도민일보
전북일보
스포츠헤드
매경뉴스
이코노스트
산업일보
CEO스코어데일리
엔아시아
아주경제
```

```
메일 목록
보낸 사람 네이버 : 새로운 환경에서 로그인 되었습니다.
보낸 사람 BitMEX : Want to Win Free Pizza and 1 Bitcoin?
보낸 사람 CGV : [CGV]개인정보 수집 출처 고지 안내
보낸 사람 Apple : 구독 확인
보낸 사람 Withings : ⚠Deleting your Withings account soon
보낸 사람 [CJ ONE] : [CJ ONE] 이용약관 개정 안내
보낸 사람 GS SHOP : [GS SHOP] 개인정보 이용내역 안내 드립니다.
보낸 사람 BitMEX : Welcome to BitMEX Options
보낸 사람 Riot Games : 라이엇 게임 계정 제재 안내
보낸 사람 키움증권 : [키움증권] 개인정보 이용내역 안내
보낸 사람 BitMEX : Want to Win Free Pizza and 1 Bitcoin?
보낸 사람 Apple : Apple에서 발행한 영수증입니다.
보낸 사람 네이버 전자문서 : [국방부] 예비군 교육훈련 소집통지서 알림
보낸 사람 noreply@mcdonalds.com : [맥도날드앱] 비밀번호 재설정 안내
보낸 사람 Google : moongye2001a@gmail.com 관련 보안 경고
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Process5 %
```